2020년 겨울계절학기

네트워크프로그래밍 실습 & 과제 #2

20. 12. 22

#2. TCP & UDP 서버

실습 #2

실습 내용

- Iterative 기반의 Server, Client 구현하기
- Echo Server 1 : TCP 문제점 살펴보기
- Echo Server 2 : TCP 문제점 해결하기
- Echo Server 3 : Connected UDP, Unconnected UDP로 Echo Server 구현하기

Iterative 기반의 Server, Client 구현하기

Server

```c
int main(int argc, char *argv[])
{

    int serv_sock, clnt_sock;
    int i;

    if (argc != 2)
    {
        printf("Usage : %s <port>\n", argv[0]);
        exit(1);
    }

    serv_sock = create_serversock(atoi(argv[1]));

    for (i = 0; i < 5; i++)
    {
        clnt_sock = create_clientsock(serv_sock);
        if (clnt_sock == -1)
            error_handling("accept() error");
        else
            printf("Connected client %d \n", i + 1);

        exchange_data(clnt_sock);
    }

    close(serv_sock);
    return 0;
}
```

Client

```c
int main(int argc, char *argv[])
{
    int sock;

    struct sockaddr_in serv_adr;

    if (argc != 3)
    {
        printf("Usage : %s <IP> <port>\n", argv[0]);
        exit(1);
    }

    sock = create_clientsock(argv[1], atoi(argv[2]));
    exchange_data(sock);
    return 0;
}
```

Iterative 기반의 Server, Client 구현하기

Server

```c
int main(int argc, char *argv[])
{

    int serv_sock, clnt_sock;
    int i;

    if (argc != 2)
    {
        printf("Usage : %s <port>\n", argv[0]);
        exit(1);
    }

    serv_sock = create_serversock(atoi(argv[1]));

    for (i = 0; i < 5; i++)
    {
        clnt_sock = create_clientsock(serv_sock);
        if (clnt_sock == -1)
            error_handling("accept() error");
        else
            printf("Connected client %d \n", i + 1);

        exchange_data(clnt_sock);
    }

    close(serv_sock);
    return 0;
}
```

```c
int create_serversock(int port)
{
    int serv_sock, clnt_sock;
    int str_len, i;

    struct sockaddr_in serv_adr;

    serv_sock = socket(PF_INET, SOCK_STREAM, 0);
    if (serv_sock == -1)
        error_handling("socket() error");

    memset(&serv_adr, 0, sizeof(serv_adr));
    serv_adr.sin_family = AF_INET;
    serv_adr.sin_addr.s_addr = htonl(INADDR_ANY);
    serv_adr.sin_port = htons(port);

    if (bind(serv_sock, (struct sockaddr *)&serv_adr, sizeof(serv_adr)) == -1)
        error_handling("bind() error");

    if (listen(serv_sock, 5) == -1)
        error_handling("listen() error");

    return serv_sock;
}
```

```c
int create_clientsock(int serv_sock){
    struct sockaddr_in clnt_adr;
    socklen_t clnt_adr_sz;
    int clnt_sock;

    clnt_adr_sz = sizeof(clnt_adr);
    clnt_sock = accept(serv_sock, (struct sockaddr *)&clnt_adr, &clnt_adr_sz);

    return clnt_sock;
}
```

```c
void exchange_data(int clnt_sock){
    int str_len;
    char message[BUF_SIZE];

    while ((str_len = read(clnt_sock, message, BUF_SIZE)) != 0){
        write(clnt_sock, message, str_len);
    }

    close(clnt_sock);
}
```

Iterative 기반의 Server, Client 구현하기

Server

```c
int main(int argc, char *argv[])
{

    int serv_sock, clnt_sock;
    int i;

    if (argc != 2)
    {
        printf("Usage : %s <port>\n", argv[0]);
        exit(1);
    }

    serv_sock = create_serversock(atoi(argv[1]));

    for (i = 0; i < 5; i++)
    {
        clnt_sock = create_clientsock(serv_sock);
        if (clnt_sock == -1)
            error_handling("accept() error");
        else
            printf("Connected client %d \n", i + 1);

        exchange_data(clnt_sock);
    }

    close(serv_sock);
    return 0;
}
```

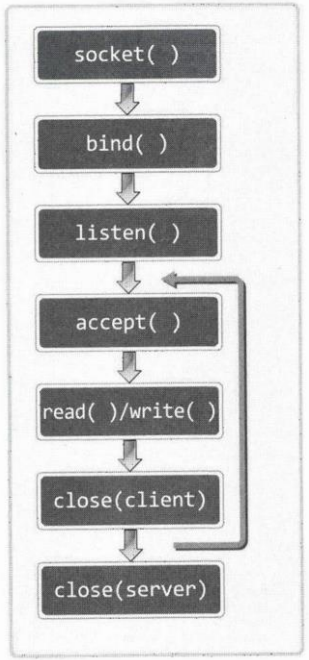


그림 04-11 : Iteratvie 서버의 함수호출 순서

create_clientsock과 exchange_data 함수만 반복됨을 확인

*참고
추후 process, thread를 배우면 exchange_data 함수 부분을 child process 또는 thread에게 위임하여 concurrent server를 구현할 수 있음

Iterative 기반의 Server, Client 구현하기

- 실행 결과

  Iterative Server, Client가 정상 작동함을 확인

Server

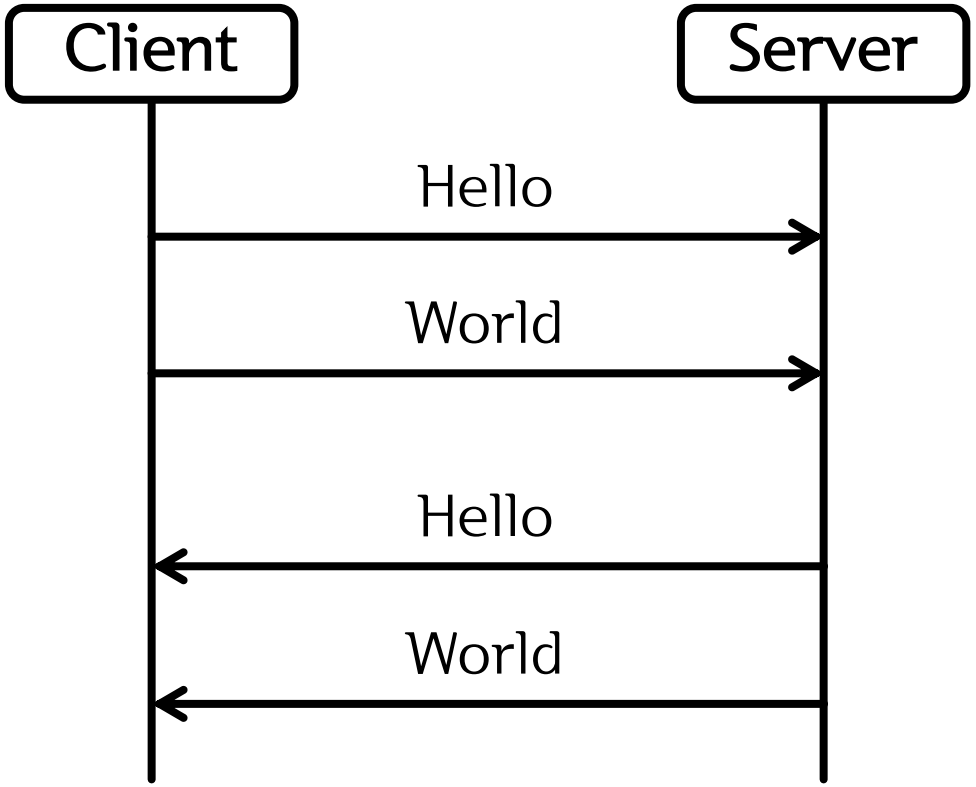
```
:/Desktop/upload#02$ gcc tcp_echoserver_without_len.c -o echoserv1
:/Desktop/upload#02$ ./echoserv1 9001
Connected client 1
^C
```

Client

```
:/Desktop/upload#02$ gcc tcp_echoclient_without_len.c -o echoclnt1
:/Desktop/upload#02$ ./echoclnt1 127.0.0.1 9001
Connected...........
Input message(Q to quit): Hello
Message from server: Hello
Input message(Q to quit): World
Message from server: World
Input message(Q to quit): Q
```

Echo Server 1 : TCP 문제점 살펴보기

- 예상 시나리오



Echo Server 1 : TCP 문제점 살펴보기

Client

```
void exchange_data2(int sock)
{
    char message[BUF_SIZE] = {
        0,
    };
    int str_len;

    write(sock, "Hello", strlen("Hello"));
    puts("Send : Hello");
    write(sock, "World", strlen("World"));
    puts("Send : World");

    str_len = read(sock, message, BUF_SIZE - 1);
    message[str_len] = 0;
    printf("Message from server: %s ", message);

    str_len = read(sock, message, BUF_SIZE - 1);
    message[str_len] = 0;
    printf("%s\n", message);

    close(sock);
}
```

메시지를 2번 전송

메시지를 2번 전송될 것이라
예상하여 read()를 2번 호출함

Echo Server 1 : TCP 문제점 살펴보기

Server

```
void exchange_data2(int clnt_sock)
{
    int str_len;
    struct sockaddr_in clnt_adr;
    socklen_t clnt_adr_sz;
    char message[BUF_SIZE];

    getchar();

    str_len = read(clnt_sock, message, BUF_SIZE);
    message[str_len] = 0;
    write(clnt_sock, message, strlen(message));
    getpeername(clnt_sock, (struct sockaddr *)&clnt_adr, &clnt_adr_sz);
    printf("receive from %s : %s\n", inet_ntoa(clnt_adr.sin_addr), message);

    str_len = read(clnt_sock, message, BUF_SIZE);
    message[str_len] = 0;
    write(clnt_sock, message, strlen(message));
    printf("receive from %s : %s\n", inet_ntoa(clnt_adr.sin_addr), message);

    close(clnt_sock);
}
```

Client 프로그램에서 메시지 2개 모두 전달했다는 프롬프트가 출력되면 enter 키를 눌러주세요.

받은 메시지를 돌려보내는 echo를 2번 수행함

Echo Server 1 : TCP 문제점 살펴보기

- 실행 결과

예상 시나리오대로 프로그램이 수행되지 않음을 확인

Server

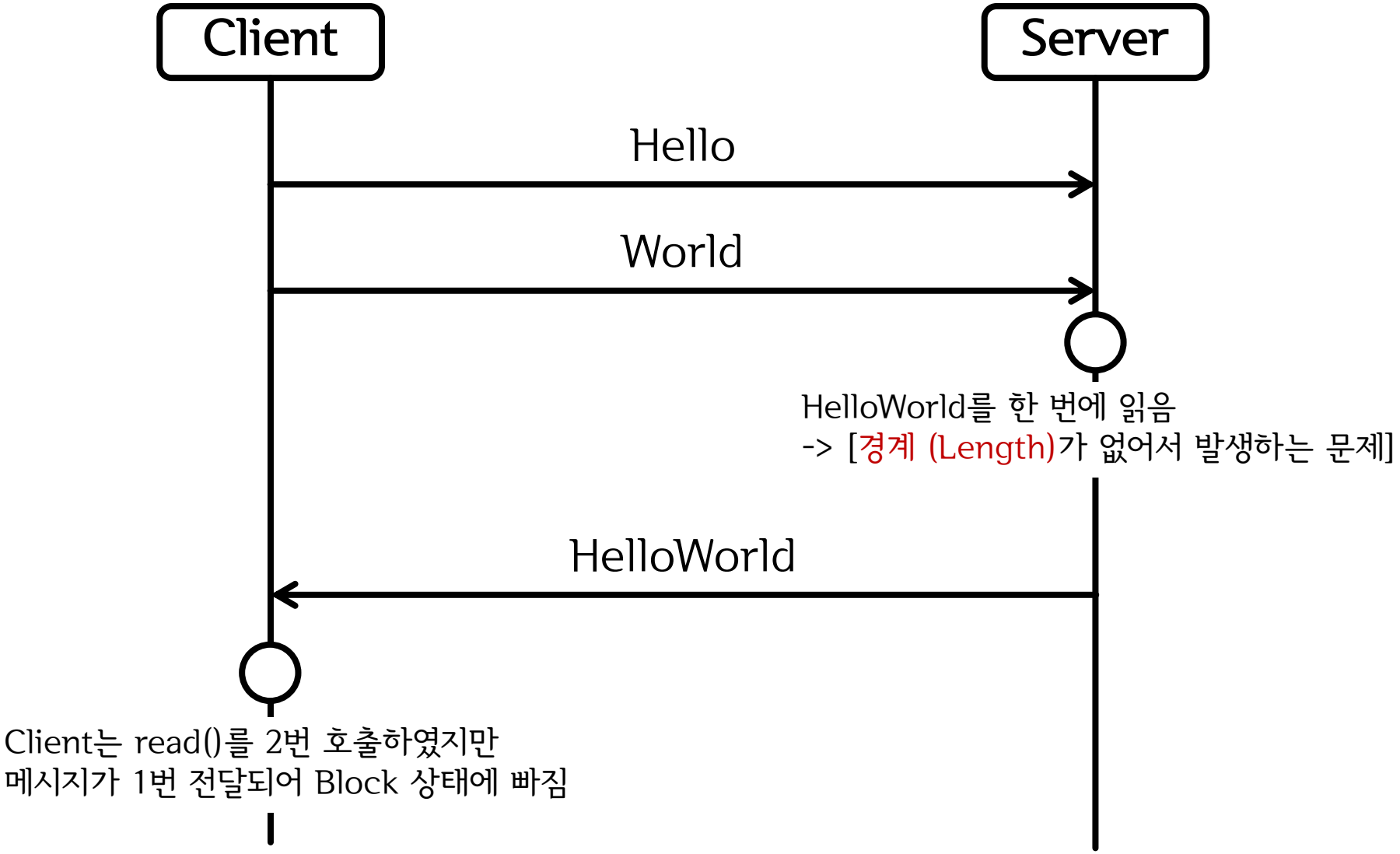
```
./tcp_echoserver_without_len 9000
Connected client 1

receive from 127.0.0.1 : HelloWorld
```

Client

```
./tcp_echoclient_without_len 127.0.0.1 9000
Connected...........
Send : Hello
Send : World
```

Echo Server 1 : TCP 문제점 살펴보기

- 실행 결과



Echo Server 2 : TCP 문제점 해결하기

- Length Field를 추가

  상대방 (peer)에게 읽어야 할 길이를 알려주자

기존 Client

```c
void exchange_data2(int sock)
{
    char message[BUF_SIZE] = {
        0,
    };
    int str_len;

    write(sock, "Hello", strlen("Hello"));
    puts("Send : Hello");
    write(sock, "World", strlen("World"));
    puts("Send : World");

    str_len = read(sock, message, BUF_SIZE - 1);
    message[str_len] = 0;
    printf("Message from server: %s ", message);

    str_len = read(sock, message, BUF_SIZE - 1);
    message[str_len] = 0;
    printf("%s\n", message);

    close(sock);
}
```

Length Field 추가

```c
void exchange_data(int sock)
{
    char message[BUF_SIZE] = {
        0,
    };
    int str_len;

    str_len = strlen("Hello");
    write(sock, &str_len, sizeof(int));
    write(sock, "Hello", str_len);
    puts("Send : Hello");

    str_len = strlen("World");
    write(sock, &str_len, sizeof(int));
    write(sock, "World", str_len);
    puts("Send : World");

    read(sock, &str_len, sizeof(int));
    str_len = read(sock, message, str_len);
    message[str_len] = 0;
    printf("Message from server: %s ", message);

    read(sock, &str_len, sizeof(int));
    str_len = read(sock, message, str_len);
    message[str_len] = 0;
    printf("%s\n", message);

    close(sock);
}
```

데이터를 전송하기 전에 str_len 값 (추후 전송할 문자열의 길이)를 먼저 알려줌

read할 때도 4bytes를 먼저 읽어서 추후 전송된 문자열의 길이만큼 읽어들임

Echo Server 2 : TCP 문제점 해결하기

실행 결과

예상 시나리오대로 프로그램이 수행됨을 확인

Server

```
./tcp_echoserver_with_len 8000
Connected client 1

receive from 0.0.0.0 : Hello
receive from 0.0.0.0 : World
```

Client

```
./tcp_echoclient_with_len 127.0.0.1 8000
Connected...........
Send : Hello
Send : World
Message from server: Hello World
```

Echo Server 3 : Connected UDP로 Echo Server 구현하기

- TCP Client와 UDP Client는 완전히 같은 절차를 가짐

Client - TCP echo main

```c
int main(int argc, char *argv[])
{
    int sock;

    if (argc != 3)
    {
        printf("Usage : %s <IP> <port>\n", argv[0]);
        exit(1);
    }

    sock = create_clientsock(argv[1], atoi(argv[2]));
    exchange_data2(sock);
    return 0;
}
```

Client - UDP echo main

```c
int main(int argc, char *argv[])
{
    int sock;
    int i;

    if (argc != 3)
    {
        printf("Usage : %s <IP> <port>\n", argv[0]);
        exit(1);
    }

    sock = create_socket(argv[1], atoi(argv[2]));
    exchange_data(sock);

    return 0;
}
```

Echo Server 3 : Connected UDP로 Echo Server 구현하기

- data 전송 방식 또한 같음

Client - TCP exchange data

```c
void exchange_data2(int sock)
{
    char message[BUF_SIZE] = {
        0,
    };
    int str_len;

    write(sock, "Hello", strlen("Hello"));
    puts("Send : Hello");
    write(sock, "World", strlen("World"));
    puts("Send : World");

    str_len = read(sock, message, BUF_SIZE - 1);
    message[str_len] = 0;
    printf("Message from server: %s ", message);

    str_len = read(sock, message, BUF_SIZE - 1);
    message[str_len] = 0;
    printf("%s\n", message);

    close(sock);
}
```

Client - UDP exchange data

```c
void exchange_data(int sock)
{
    char message[BUF_SIZE] = {
        0,
    };
    int str_len;

    write(sock, "Hello", strlen("Hello"));
    puts("Send : Hello");

    write(sock, "World", strlen("World"));
    puts("Send : World");

    str_len = read(sock, message, BUF_SIZE - 1);
    message[str_len] = 0;
    printf("Message from server: %s ", message);

    str_len = read(sock, message, BUF_SIZE - 1);
    message[str_len] = 0;
    printf("%s\n", message);

    close(sock);
}
```

Echo Server 3 : Connected UDP로 Echo Server 구현하기

Server - TCP echo main

```c
int main(int argc, char *argv[])
{

    int serv_sock, clnt_sock;
    int i;

    if (argc != 2)
    {
        printf("Usage : %s <port>\n", argv[0]);
        exit(1);
    }

    serv_sock = create_serversock(atoi(argv[1]));

    for (i = 0; i < 5; i++)
    {
        clnt_sock = create_clientsock(serv_sock);
        if (clnt_sock == -1)
            error_handling("accept() error");
        else
            printf("Connected client %d \n", i + 1);

        exchange_data(clnt_sock);
    }

    close(serv_sock);
    return 0;
}
```

Server - UDP echo main

```c
int main(int argc, char *argv[])
{
    int sock;
    int i;

    if (argc != 2)
    {
        printf("Usage : %s <port>\n", argv[0]);
        exit(1);
    }

    sock = create_socket(atoi(argv[1]));

    for (i = 0; i < 5; i++)
        exchange_data(sock);

    close(sock);
    return 0;
}
```

UDP는 연결이라는 개념이 없기 때문에 연결 설정 역할을 수행하는 Server Socket과 데이터 전송을 위한 Client Socket이 구분되어 있지 않음

Echo Server 3 : Connected UDP로 Echo Server 구현하기

Server - create_socket

listen 호출을 제외한 나머지 부분은 같은 절차를 가짐

```
int create_socket(int port)
{
    int sock;
    struct sockaddr_in my_adr;

    sock = socket(PF_INET, SOCK_DGRAM, 0);
    if (sock == -1)
        error_handling("socket() error");

    memset(&my_adr, 0, sizeof(my_adr));
    my_adr.sin_family = AF_INET;
    my_adr.sin_addr.s_addr = htonl(INADDR_ANY);
    my_adr.sin_port = htons(port);

    if (bind(sock, (struct sockaddr *)&my_adr, sizeof(my_adr)) == -1)
        error_handling("bind() error");

    return sock;
}
```

Echo Server 3 : Connected UDP로 Echo Server 구현하기

- TCP, UDP 두 프로그램에서 데이터를 읽어들여 그대로 전송하는 메커니즘이 똑같음을 확인할 수 있음
- 두 코드 모두 length와 같은 여분의 메타데이터를 전송하지 않음

Server - TCP exchange data

```c
void exchange_data2(int clnt_sock)
{
    int str_len;
    struct sockaddr_in clnt_adr;
    socklen_t clnt_adr_sz;
    char message[BUF_SIZE];

    getchar();

    str_len = read(clnt_sock, message, BUF_SIZE);
    message[str_len] = 0;
    write(clnt_sock, message, strlen(message));
    getpeername(clnt_sock, (struct sockaddr *)&clnt_adr, &clnt_adr_sz);
    printf("receive from %s : %s\n", inet_ntoa(clnt_adr.sin_addr), message);

    str_len = read(clnt_sock, message, BUF_SIZE);
    message[str_len] = 0;
    write(clnt_sock, message, strlen(message));
    printf("receive from %s : %s\n", inet_ntoa(clnt_adr.sin_addr), message);

    close(clnt_sock);
}
```

Server - UDP exchange data

```c
void exchange_data(int sock)
{
    int str_len;
    char message[BUF_SIZE];
    socklen_t clnt_adr_sz;
    struct sockaddr_in clnt_adr;

    getchar();

    clnt_adr_sz = sizeof(clnt_adr);
    str_len = recvfrom(sock, message, BUF_SIZE, 0, (struct sockaddr *)&clnt_adr, &clnt_adr_sz);
    message[str_len] = 0;
    printf("receive from %s : %s\n", inet_ntoa(clnt_adr.sin_addr), message);
    sendto(sock, message, str_len, 0, (struct sockaddr *)&clnt_adr, clnt_adr_sz);

    clnt_adr_sz = sizeof(clnt_adr);
    str_len = recvfrom(sock, message, BUF_SIZE, 0, (struct sockaddr *)&clnt_adr, &clnt_adr_sz);
    message[str_len] = 0;
    printf("receive from %s : %s\n", inet_ntoa(clnt_adr.sin_addr), message);
    sendto(sock, message, str_len, 0, (struct sockaddr *)&clnt_adr, clnt_adr_sz);
}
```

Echo Server 4 : Unconnected UDP로 Echo Server 구현하기

Client - Connected UDP create_socket

```c
int create_socket(char *ip, int port)
{
    int sock;
    char message[BUF_SIZE];
    int str_len;
    socklen_t adr_sz;

    struct sockaddr_in serv_adr, from_adr;

    sock = socket(PF_INET, SOCK_DGRAM, 0);
    if (sock == -1)
        error_handling("socket() error");

    memset(&serv_adr, 0, sizeof(serv_adr));
    serv_adr.sin_family = AF_INET;
    serv_adr.sin_addr.s_addr = inet_addr(ip);
    serv_adr.sin_port = htons(port);

    connect(sock, (struct sockaddr *)&serv_adr, sizeof(serv_adr));

    return sock;
}
```

Client - Unconnected UDP create_socket

```c
int create_socket()
{
    int sock;
    char message[BUF_SIZE];
    int str_len;

    sock = socket(PF_INET, SOCK_DGRAM, 0);
    if (sock == -1)
        error_handling("socket() error");

    return sock;
}
```

- Unconnected UDP에서는 연결을 위한 connect 함수 호출과 connect 함수 호출에 전달할 주소 객체 생성을 위한 코드가 사라진 것을 확인할 수 있음

Echo Server 4 : Unconnected UDP로 Echo Server 구현하기

- Unconnected UDP Socket은 상대방이 정해져 있지 않기 때문에 sendto 함수가 사용됨
- sendoto 함수에는 **상대방의 주소**가 함께 전달되어야 함

Client - Connected UDP exchange_data

```c
void exchange_data(int sock)
{
    char message[BUF_SIZE] = {
        0,
    };
    int str_len;

    write(sock, "Hello", strlen("Hello"));
    puts("Send : Hello");

    write(sock, "World", strlen("World"));
    puts("Send : World");

    str_len = read(sock, message, BUF_SIZE - 1);
    message[str_len] = 0;
    printf("Message from server: %s ", message);

    str_len = read(sock, message, BUF_SIZE - 1);
    message[str_len] = 0;
    printf("%s\n", message);

    close(sock);
}
```

Client - Unconnected UDP exchange_data

```c
void exchange_data(int sock, char *ip, int port)
{
    struct sockaddr_in serv_adr, from_adr;
    socklen_t adr_sz;
    int str_len;
    char message[BUF_SIZE] = {
        0,
    };

    adr_sz = sizeof(serv_adr);
    memset(&serv_adr, 0, sizeof(serv_adr));
    serv_adr.sin_family = AF_INET;
    serv_adr.sin_addr.s_addr = inet_addr(ip);
    serv_adr.sin_port = htons(port);

    sendto(sock, "Hello", strlen("Hello"), 0, (struct sockaddr *)&serv_adr, adr_sz);
    puts("Send : Hello");

    sendto(sock, "World", strlen("World"), 0, (struct sockaddr *)&serv_adr, adr_sz);
    puts("Send : World");

    str_len = read(sock, message, BUF_SIZE - 1);
    message[str_len] = 0;
    printf("Message from server: %s ", message);

    str_len = read(sock, message, BUF_SIZE - 1);
    message[str_len] = 0;
    printf("%s\n", message);

    close(sock);
}
```

Echo Server 4 : Unconnected UDP로 Echo Server 구현하기

- 실행 결과

connected_udpecho_client, unconnected_udpecho_client, 모두 다 잘 실행됨을 확인

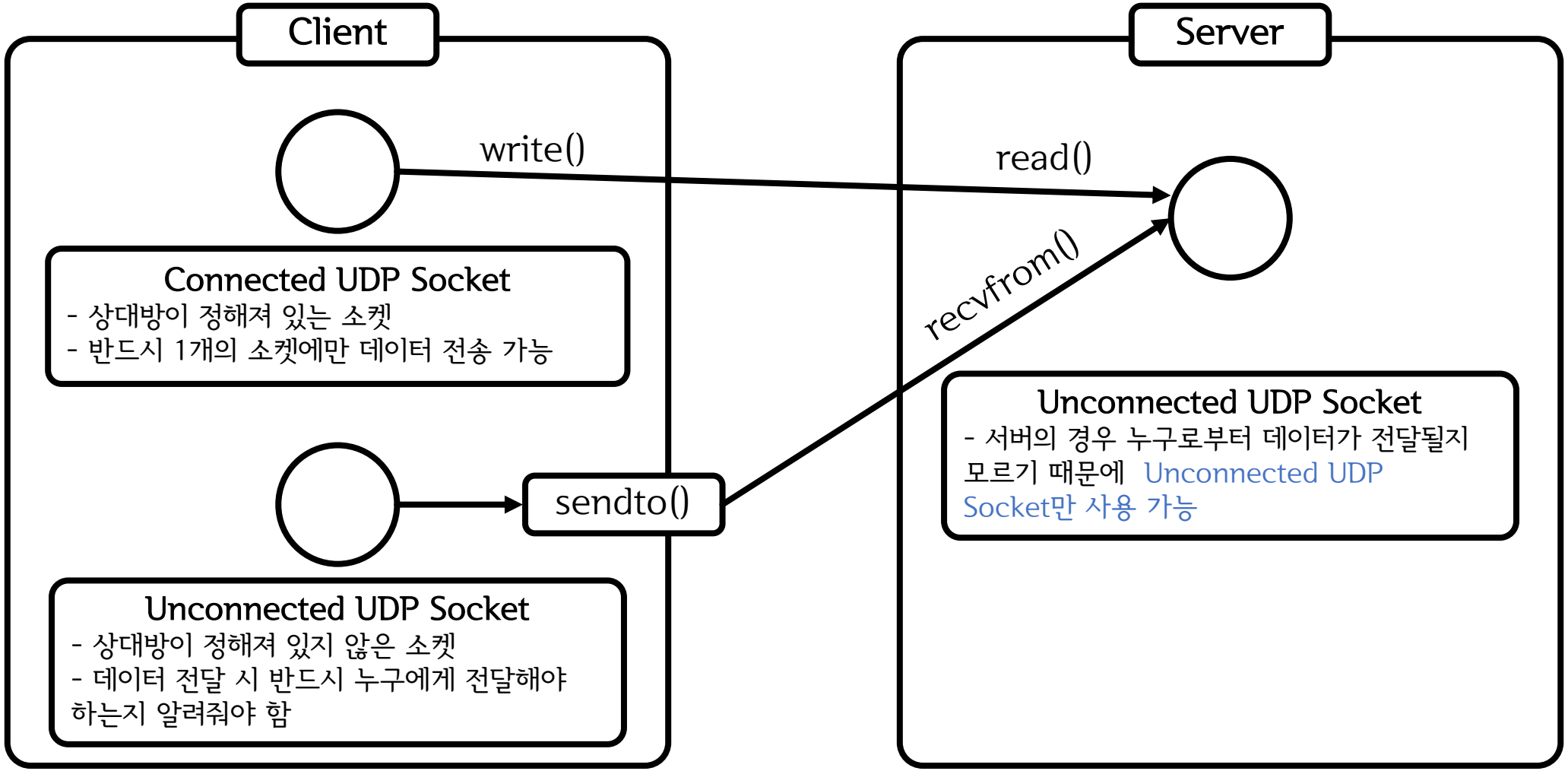
Server

```
./udpecho_server 9000

receive from 127.0.0.1 : Hello
receive from 127.0.0.1 : World

receive from 127.0.0.1 : Hello
receive from 127.0.0.1 : World
```

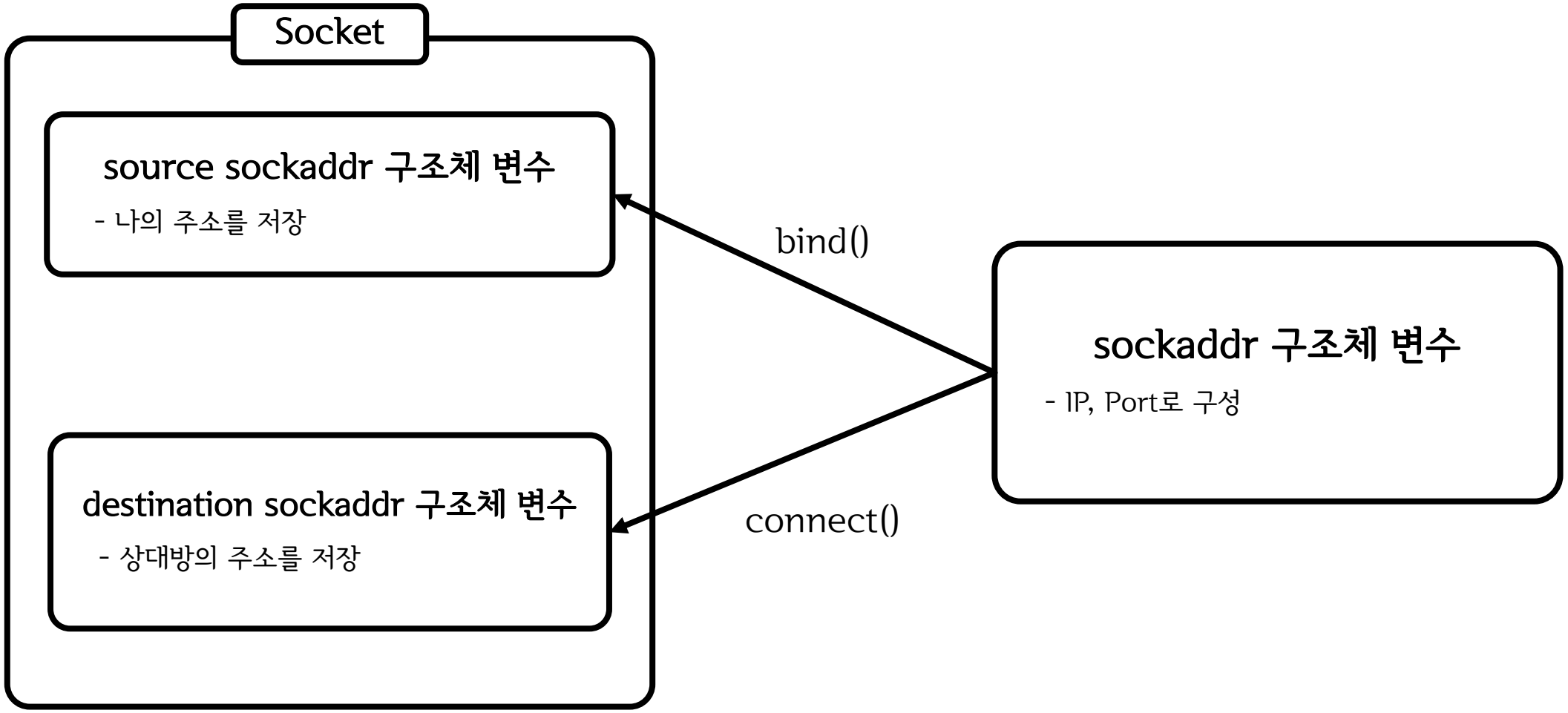
Client

```
./connected_udpecho_client 127.0.0.1 9000
Send : Hello
Send : World
Message from server: Hello World
./unconnected_udpecho_client 127.0.0.1 9000
Send : Hello
Send : World
Message from server: Hello World
```

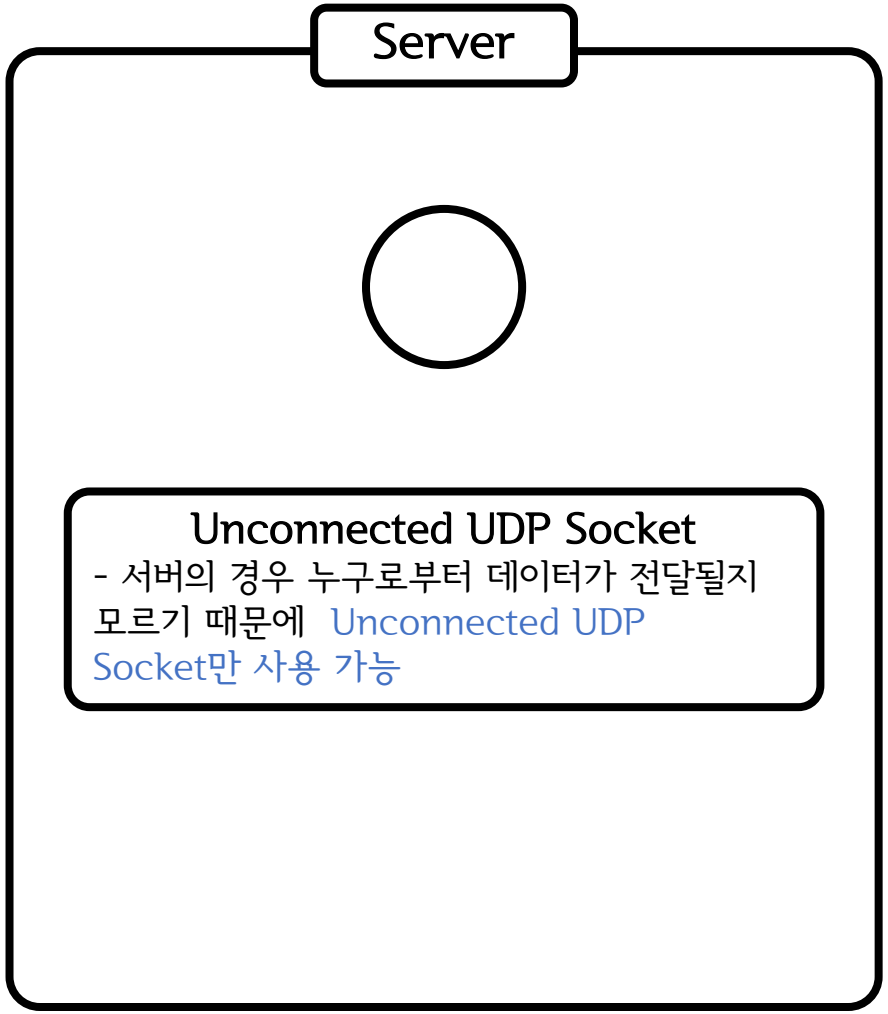
Connected UDP Socket과 Unconnected UDP Socket



Connected UDP Socket과 Unconnected UDP Socket



Connected UDP Socket과 Unconnected UDP Socket



- Unconnected UDP Socket은 상대방이 결정되어 있지 않아 누구로부터 데이터가 전달되었는지 모름
- Read() 대신에 recvfrom()을 통해 데이터를 수신해야 함
- recvfrom()은 프로그램에게 누구로부터 전달된 메시지인지를 알려주기 위해 주소정보를 저장할 객체를 파라미터로 받아야 함

```
clnt_adr_sz = sizeof(clnt_adr);
str_len = recvfrom(sock, message, BUF_SIZE, 0, (struct sockaddr *)&clnt_adr, &clnt_adr_sz);
message[str_len] = 0;
printf("receive from %s : %s\n", inet_ntoa(clnt_adr.sin_addr), message);
sendto(sock, message, str_len, 0, (struct sockaddr *)&clnt_adr, clnt_adr_sz);
```

이와 같은 형태로 데이터를 전송한 상대방 확인 가능

누구로부터 전송된 데이터인지 확인 하기 위한 주소 구조체 변수

Connected UDP Socket과 Unconnected UDP Socket

- Connected UDP Socket 또는 TCP 같이 상대방이 결정되어 있는 소켓은 getpeername() 과 소켓 파일 디스트립터로 상대방의 주소를 확인할 수 있음
- (해당 내용은 소켓 옵션을 소개하는 챕터에서 자세히 설명됨)

```
getpeername(clnt_sock, (struct sockaddr *)&clnt_adr, &clnt_adr_sz);
printf("receive from %s : %s\n", inet_ntoa(clnt_adr.sin_addr), message);
```

과제 #02

파일 내용을 전송하는 Server-Client 코드 작성

- ‘sample-mp4-file.mp4’ 파일 생성
  - 아래 명령어를 통해 ‘sample-mp4-file.mp4’ 파일을 다운로드
  - **파일 다운로드)** wget https://www.learningcontainer.com/wp-content/uploads/2020/05/sample-mp4-file.mp4

- ‘hw02_server.c’ server 프로그램 생성
  - Server가 실행되면 ‘smaple-mp4-file.mp4’ 파일의 내용을 읽음
    (*주의: argv를 통해 파일명을 받아야 함. 소스 코드에 ‘sample-mp4-file.mp4’ 파일명이 언급되면 감점 처리)
    실행 예시) ./hw02_server sample-mp4-file.mp4 9000
  - Server는 1000 bytes 크기의 버퍼를 사용하여 파일을 읽어들여 Client에게 전달함
    - 파일을 모두 전송할 때까지 Socket을 통해 Client에 파일을 전송함
    read() 함수로 1000 bytes씩 읽고, write() 함수를 통해 1000 bytes씩 전송
    전송 전에 8 bytes 정수를 통해 파일 크기를 미리 Client에 전달함 (파일 크기를 확인하는 코드는 제공 예정)
  - 모든 내용 전달 후, Client로부터 ‘Thank you’ 메시지를 수신하고 소켓 close()

과제 #02

- **'hw02_client.c' client 프로그램 생성**
  - Client가 실행되면 Server로부터 받은 내용을 저장할 파일명과 **함께** 입력 받음
    파일명은 **'video.mp4'**로 통일, **만약 video.mp4 파일이 없다면 생성하는 옵션 설정**
    (**권한표시 옵션을 644로 설정 (sudo chmod 644 video.mp4))
    (*주의: argv를 통해 파일명을 받아야 함. 소스 코드에 'video.mp4' 파일명이 언급되면 감점 처리)
    실행 예시) ./hw02_client video.mp4 127.0.0.1 9000
  - Client는 입력 받은 IP, Port로 Server에 접속
  - Client와 Server가 연결되면, Server로부터 **8 bytes**를 읽고 파일의 사이즈를 획득함
  - 획득한 사이즈만큼 **70 bytes** 배열을 사용하여 읽어온 후 파일에 저장함
  - 모든 내용 전달 후, Server로 'Thank you' 문자열을 전달하고 소켓 close()

- **주의 사항:** read, write 함수에 전달하는 파라미터를 배열이 아닌 char 변수를 사용할 것!
- **제출 파일:** hw02_server.c, hw02_client.c, 실행 결과 스크린샷 이미지 (jpg, png 등)
- **제출 방법:** '학번_이름_HW02.zip'으로 압축하여 LMS 과제 게시판에 제출
- **제출 기한:** 2020년 12월 28일 (월) 13:59까지

- 실행 결과 스크린샷 이미지 예시

Server Program

```
➜ hw02              ✗ ls
hw02_client  hw02_client.c  hw02_server  hw02_server.c  sample-mp4-file.mp4
➜ hw02              ✗ ./hw02_server sample-mp4-file.mp4 9000
Connected client 1
Thank you
^C
➜ hw02              ✗ ls
hw02_client    hw02_server    sample-mp4-file.mp4
hw02_client.c  hw02_server.c  video.mp4
➜ hw02              ✗ diff sample-mp4-file.mp4 video.mp4
diff: video.mp4: 허가 거부
➜ hw02              ✗ chmod 664 video.mp4
➜ hw02              ✗ diff sample-mp4-file.mp4 video.mp4
➜ hw02              ✗
```

Client Program

```
➜ hw02              ✗ ./hw02_client video.mp4 127.0.0.1 9000
Connected...........
Start file download [filesize : 10546620]
--Complete file download--
Send : Thank you
➜ hw02              ✗
```